Trabalho Prático: Árvore Geradora Mínima

Implementação do Algoritmo de Kruskal com Cytoscape.js

Objetivo do Trabalho

O objetivo deste trabalho prático é desenvolver uma aplicação web interativa que permita a visualização passo a passo do **Algoritmo de Kruskal** para encontrar a Árvore Geradora Mínima (MST - *Minimum Spanning Tree*) de um grafo. Os alunos deverão utilizar a biblioteca **Cytoscape.js** para a renderização e manipulação visual do grafo.

Requisitos da Aplicação

A aplicação web deve possuir uma interface dividida em duas partes principais: um painel de desenho do grafo e uma barra lateral para exibição do log de execução.

A implementação deve satisfazer os seguintes requisitos:

1. **Criação de Nós e Arestas:** Deve ser possível adicionar nós clicando no painel. Arestas devem ser criadas selecionando um nó de origem e, em seguida, um nó de destino.
2. **Pesos Automáticos e Manuais:** Ao criar uma aresta, seu peso padrão deve ser calculado como a distância euclidiana entre os dois nós. O usuário também deve poder alterar o peso de qualquer aresta clicando sobre ela, através de uma caixa de diálogo (*prompt*).
3. **Exclusão de Elementos:** Nós e arestas devem poder ser excluídos ao se clicar com o botão direito (ou toque longo) sobre eles. Ao excluir um nó, todas as suas arestas incidentes também devem ser removidas automaticamente.
4. **Algoritmo de Kruskal Passo a Passo:**
   - A aplicação não deve apenas chamar a função pronta de Kruskal do Cytoscape para resolver o problema, mas sim implementar a lógica passo a passo, utilizando a estrutura de dados de **Conjuntos Disjuntos** (Union-Find).
   - Ao clicar em um botão "Executar Kruskal", a aplicação deve processar as arestas do grafo em ordem crescente de seus pesos.
5. **Log de Execução (Barra Lateral):** O sistema deve possuir uma barra lateral que apresente as iterações do algoritmo:
   - Exibir o conjunto de arestas da árvore (A) no início (vazio) e seu estado em cada iteração.
   - Exibir os conjuntos auxiliares de vértices (do algoritmo Union-Find) no início e como eles se unem a cada iteração.
   - O detalhamento de cada aresta analisada, informando claramente se a aresta foi adicionada ao conjunto A (pois os vértices estavam em componentes distintos) ou se foi descartada (pois formaria um ciclo).

Exemplos Visuais da Interface

Abaixo encontram-se exemplos visuais da interface esperada, mostrando a área de modelagem do grafo e os logs de iteração do algoritmo na barra lateral.

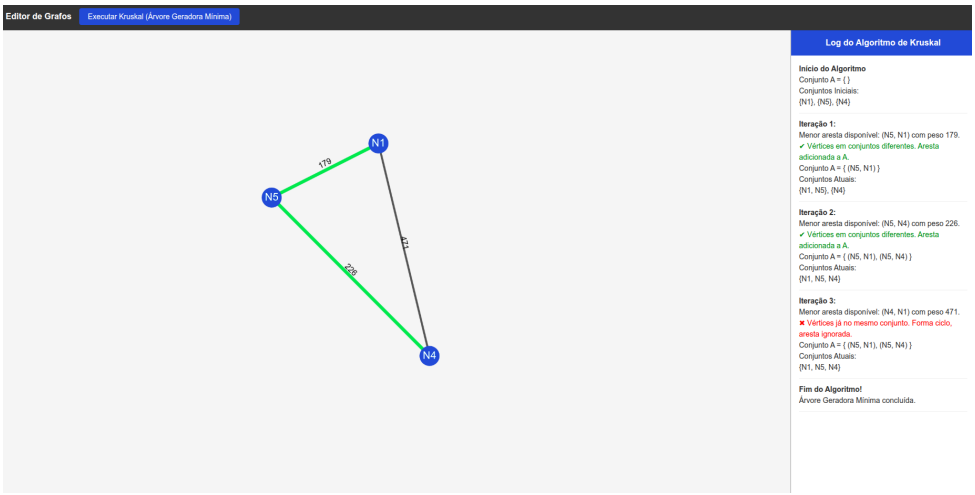


Figura 1: Exemplo da interface com o grafo construído sendo editado pelo usuário.

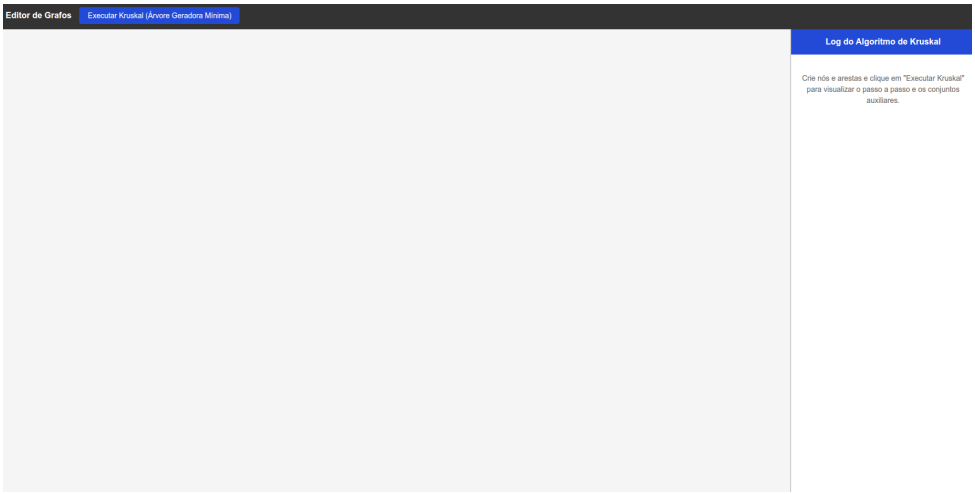


Figura 2: Execução do Algoritmo de Kruskal, exibindo a Árvore Geradora Mínima destacada no grafo.

Exemplo do Log de Execução

Para garantir que a formatação da barra lateral atenda aos requisitos, a saída gerada a cada execução deve seguir o formato textual exemplificado abaixo (a interface real pode usar estilos HTML/CSS para deixar mais legível, com cores indicando sucesso ou falha).

```
Início do Algoritmo
Conjunto A = { }
Conjuntos Iniciais:
```

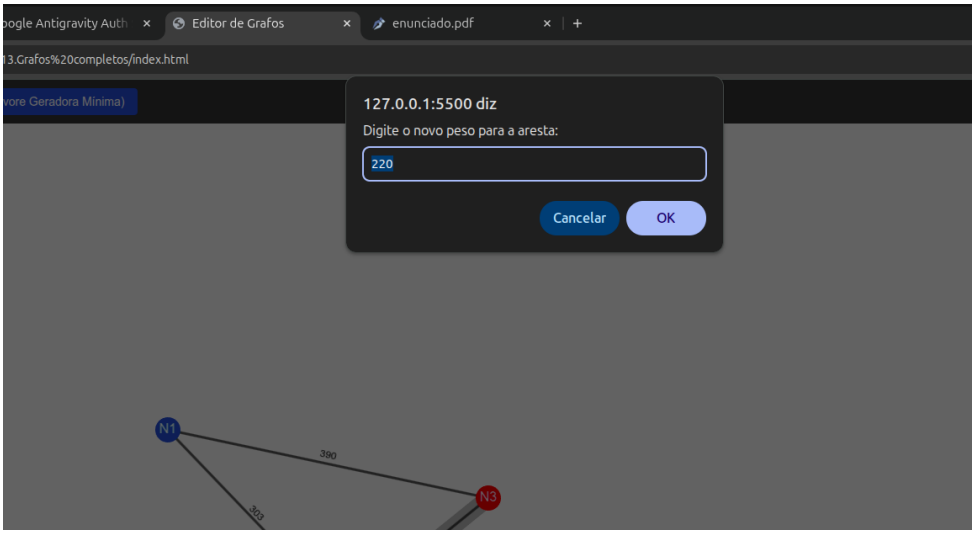


Figura 3: Edição manual do peso de uma aresta após clique no elemento.

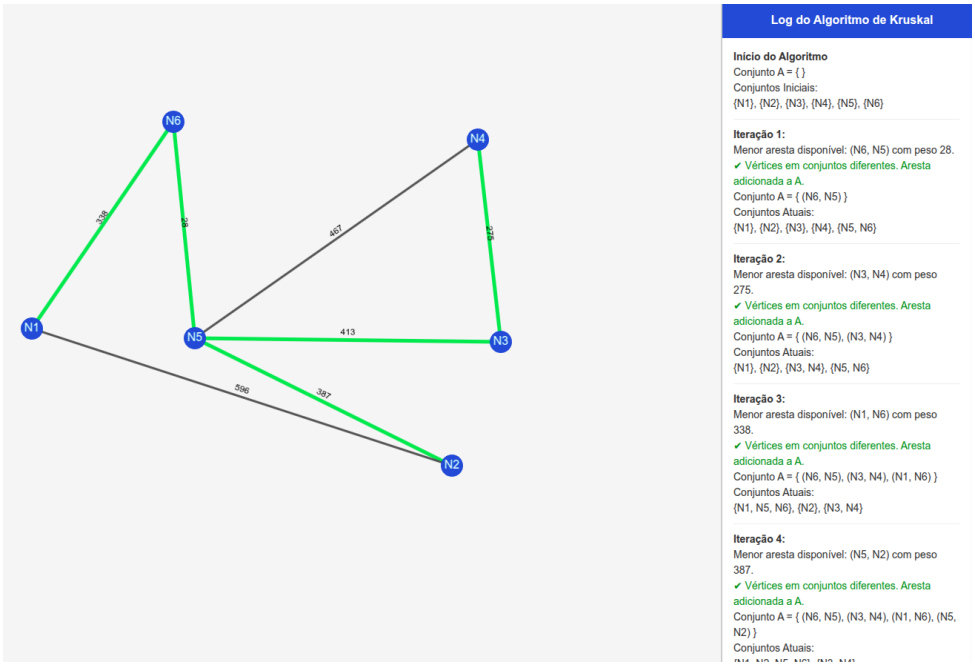


Figura 4: Outra parte da execução, mostrando os conjuntos disjuntos e o conjunto A passo a passo.

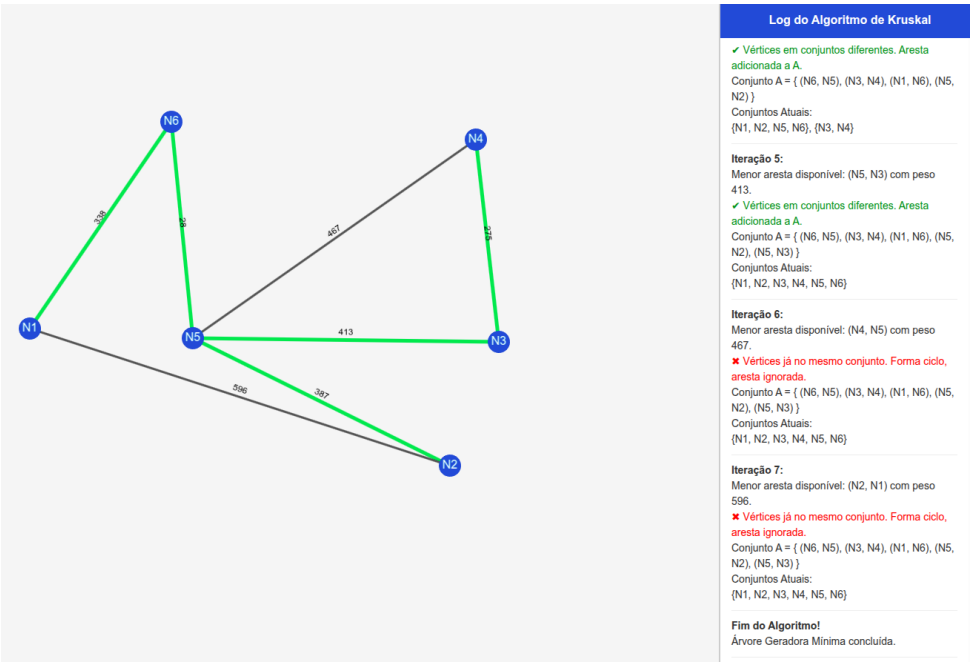


Figura 5: Detalhe do log de execução concluído.

```
{N1}, {N2}, {N3}, {N4}, {N5}, {N6}

Iteração 1:
Menor aresta disponível: (N6, N5) com peso 28.
 Vértices em conjuntos diferentes. Aresta adicionada a A.
Conjunto A = { (N6, N5) }
Conjuntos Atuais:
{N1}, {N2}, {N3}, {N4}, {N5, N6}

Iteração 2:
Menor aresta disponível: (N3, N4) com peso 275.
 Vértices em conjuntos diferentes. Aresta adicionada a A.
Conjunto A = { (N6, N5), (N3, N4) }
Conjuntos Atuais:
{N1}, {N2}, {N3, N4}, {N5, N6}

Iteração 3:
Menor aresta disponível: (N1, N6) com peso 338.
 Vértices em conjuntos diferentes. Aresta adicionada a A.
Conjunto A = { (N6, N5), (N3, N4), (N1, N6) }
Conjuntos Atuais:
{N1, N5, N6}, {N2}, {N3, N4}

Iteração 4:
Menor aresta disponível: (N5, N2) com peso 387.
 Vértices em conjuntos diferentes. Aresta adicionada a A.
Conjunto A = { (N6, N5), (N3, N4), (N1, N6), (N5, N2) }
Conjuntos Atuais:
```

```
{N1, N2, N5, N6}, {N3, N4}

Iteração 5:
Menor aresta disponível: (N5, N3) com peso 413.
 Vértices em conjuntos diferentes. Aresta adicionada a A.
Conjunto A = { (N6, N5), (N3, N4), (N1, N6), (N5, N2), (N5, N3) }
Conjuntos Atuais:
{N1, N2, N3, N4, N5, N6}

Iteração 6:
Menor aresta disponível: (N4, N5) com peso 467.
 Vértices já no mesmo conjunto. Forma ciclo, aresta ignorada.
Conjunto A = { (N6, N5), (N3, N4), (N1, N6), (N5, N2), (N5, N3) }
Conjuntos Atuais:
{N1, N2, N3, N4, N5, N6}

Iteração 7:
Menor aresta disponível: (N2, N1) com peso 596.
 Vértices já no mesmo conjunto. Forma ciclo, aresta ignorada.
Conjunto A = { (N6, N5), (N3, N4), (N1, N6), (N5, N2), (N5, N3) }
Conjuntos Atuais:
{N1, N2, N3, N4, N5, N6}

Fim do Algoritmo!
Árvore Geradora Mínima concluída.
```

Entrega: Vídeo Explicativo

Além do código-fonte, o aluno deverá gravar e entregar um **vídeo explicativo** demonstrando o funcionamento da aplicação. O vídeo deve cobrir obrigatoriamente os seguintes casos:

1. **Grafo Completo** K_5 – grafo com 5 vértices onde todo par de vértices possui uma aresta (10 arestas no total). O aluno deve construir o grafo, atribuir pesos e executar o Kruskal, explicando cada iteração do log.

2. **Grafo Bipartido Completo** $K_{3,3}$ – grafo com dois grupos de 3 vértices, onde cada vértice de um grupo está ligado a todos os vértices do outro grupo (9 arestas). O aluno deve demonstrar a execução do Kruskal destacando a estrutura bipartida.

3. **Grafo com Duas Componentes Conexas** – grafo em que os vértices estão divididos em dois grupos desconectados entre si. O aluno deve demonstrar que o algoritmo de Kruskal produz **florestas geradoras mínimas**, uma por componente, e explicar o comportamento do log nesse caso.

4. **Grafo Completo** K_4 – grafo com 4 vértices e todos conectados entre si (6 arestas). O aluno deve executar o Kruskal e identificar no log quais arestas foram ignoradas por formarem ciclos.

O vídeo deve ter no mínimo **5 minutos** e o aluno deve narrar em voz própria, explicando os conceitos do algoritmo enquanto demonstra cada grafo na aplicação.

Wireframes das Telas Esperadas

As figuras abaixo apresentam os wireframes das telas que devem ser reproduzidas no vídeo, ilustrando a estrutura de cada grafo e o estado esperado da interface.

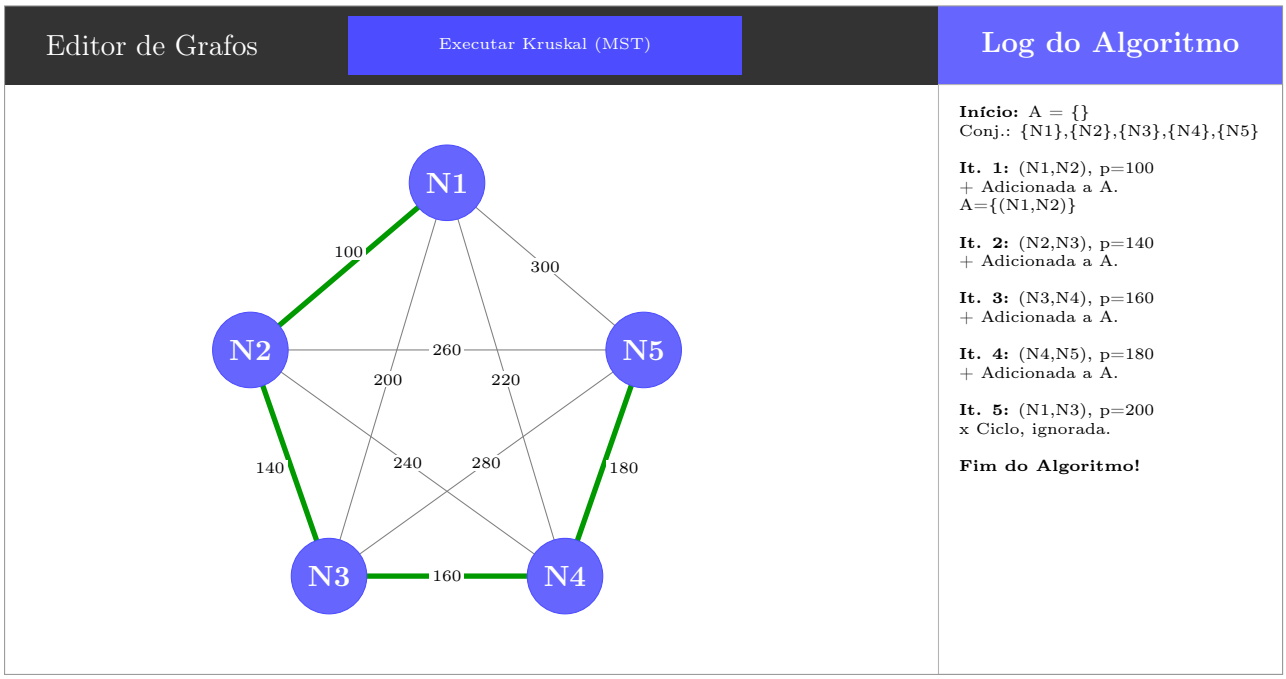


Figura 6: Wireframe – Grafo Completo K_5 com MST destacada (arestas verdes).

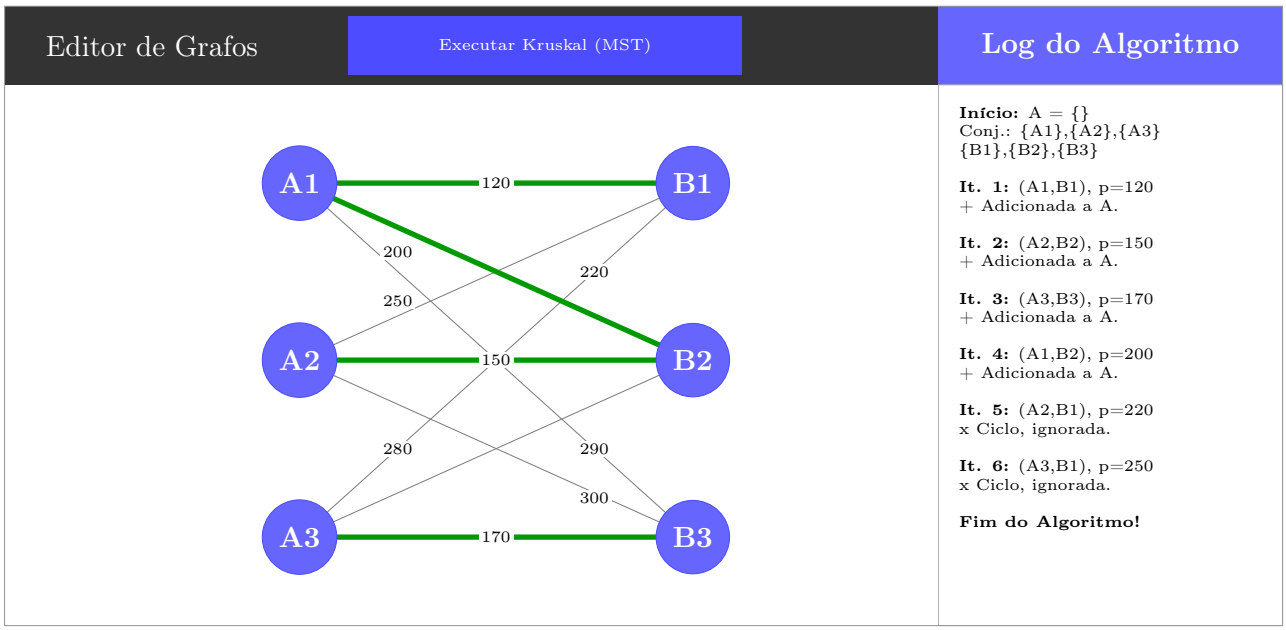


Figura 7: Wireframe – Grafo Bipartido Completo $K_{3,3}$ com MST destacada.

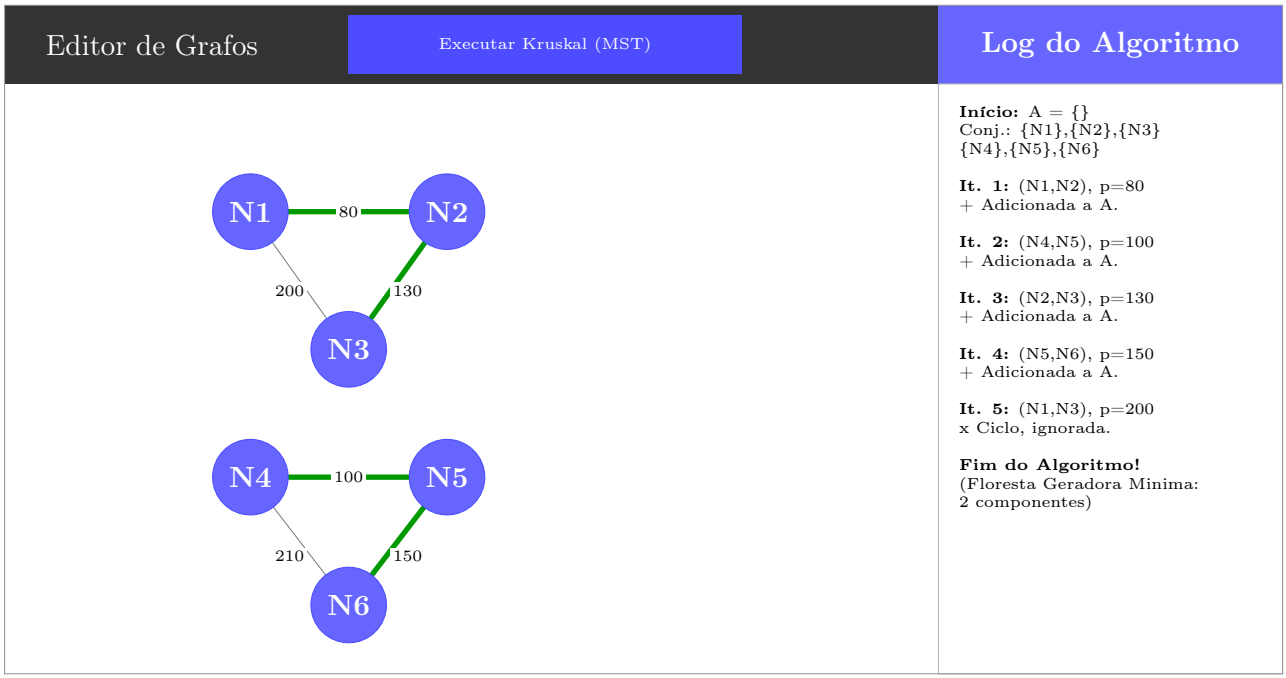


Figura 8: Wireframe – Grafo com Duas Componentes Conexas. O Kruskal produz uma Floresta Geradora Mínima (uma MST por componente).

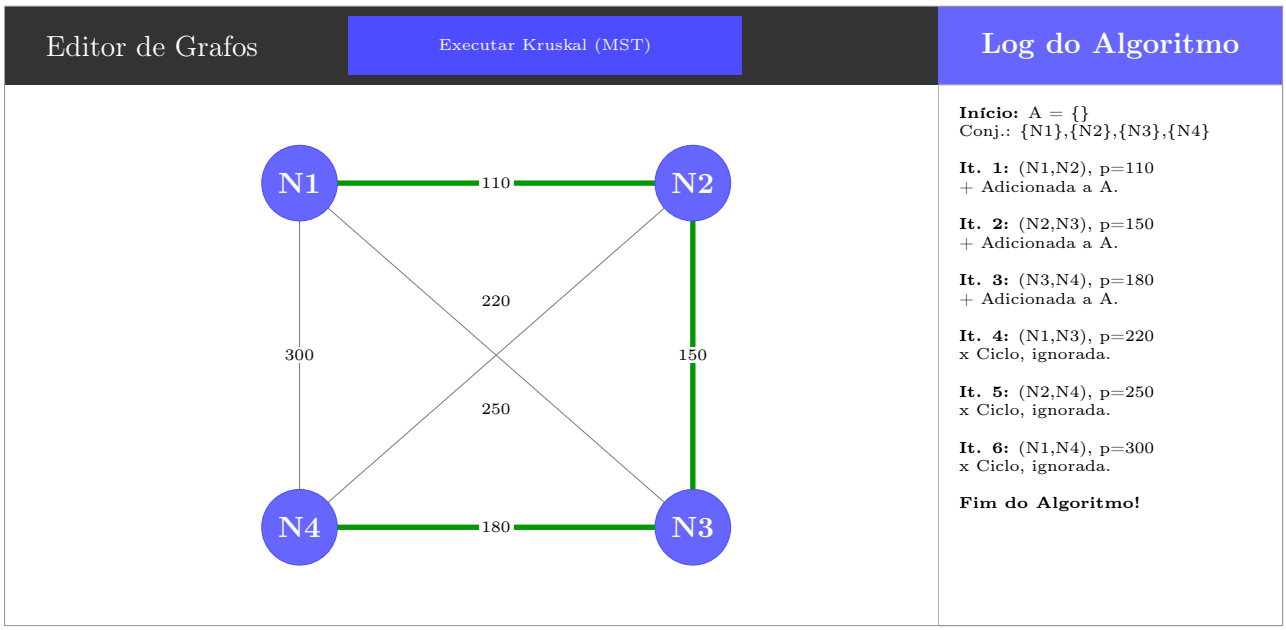


Figura 9: Wireframe – Grafo Completo K_4. As 3 arestas da MST são destacadas; as 3 diagonais formam ciclo e são ignoradas.

Critérios de Avaliação

O trabalho será avaliado de acordo com os seguintes critérios:

- Corretude da implementação do algoritmo de Kruskal e do Union-Find.
- Funcionamento adequado de todas as interações e manipulações no Cytoscape.js (adição, edição de pesos e exclusão).
- Clareza e detalhamento na formatação do log gerado na barra lateral (apresentando a evolução do conjunto A e dos conjuntos disjuntos).
- Interface amigável, feedback visual adequado (como destaque das arestas que pertencem à MST) e código fonte bem organizado.
- **Vídeo explicativo** cobrindo os 4 grafos obrigatórios (K_5, $K_{3,3}$, duas componentes e K_4), com narração própria explicando cada iteração.
- **Submissão do código fonte** no Google Classroom, até o dia 26/06/2026, as 23h59.